PRACTICAL – 9

Title: Implement more complex Apex triggers or classes to address specific business requirements. Test and debug Apex code using Salesforce Developer Console

Aim: To implement advanced Apex logic using an Apex class and trigger to handle business rules on Student records and to test and debug the code using Salesforce Developer Console.

Software / Tools Required

- Web Browser (Google Chrome recommended)
- Internet Connection
- Salesforce Developer Edition Account

Theory: As Salesforce applications grow in complexity, business logic should be organized using **Apex classes** instead of placing all logic inside triggers.
A recommended practice is:

- Keep triggers simple
- Move logic to Apex classes
- Reuse logic across multiple triggers or processes

The **Developer Console** is used to:

- Execute Apex code
- View debug logs
- Identify runtime errors

Practical Exercise

Step 1: Define Business Requirement

Business Rule for Virtual University:

- If a Student has an Email address, mark the student as **Verified**
- If a Student does not have an Email address, prevent verification

This logic must be reusable and maintainable.

Step 2: Create an Apex Class for Business Logic

1. Go to **Setup**.
2. Search for **Apex Classes**.
3. Click **New**.
4. Enter the following Apex code:
```
public class StudentVerificationHandler {

    public static void verifyStudents(List<Student__c> students) {
        for (Student__c s : students) {
            if (s.Email__c != null && s.Email__c != '') {
                s.Verified__c = true;
            } else {
                s.Verified__c = false;
            }
        }
    }
}
```
5. Click **Save**.

Ensure the class saves without errors.

Step 3: Modify Apex Trigger to Use Apex Class

1. Go to **Setup**.
2. Open **Apex Triggers**.
3. Open the existing **StudentVerifyTrigger**.
4. Replace the trigger code with:
```
trigger StudentVerifyTrigger on Student__c (before insert, before update) {
    StudentVerificationHandler.verifyStudents(Trigger.new);
}
```
5. Click **Save**.

Step 4: Test the Updated Trigger

1. Open the **Students** tab.
2. Create a new Student record with Email.
3. Save the record and verify that **Verified** is checked.
4. Create another Student record without Email.
5. Save the record and verify that **Verified** remains unchecked.

Step 5: Open Developer Console

1. Click the **Gear icon**.
2. Select **Developer Console**.
3. The Developer Console window opens.

Step 6: Execute Apex Code in Developer Console

1. In Developer Console, click **Debug → Open Execute Anonymous Window**.
2. Enter the following code:
```
List<Student__c> students = [SELECT Id, Email__c, Verified__c FROM Student__c LIMIT 5];
StudentVerificationHandler.verifyStudents(students);
update students;
```
3. Click **Execute**.

Step 7: View Debug Logs

1. In Developer Console, click **Logs** tab.
2. Open the latest log.
3. Observe:
    - Trigger execution
    - Class method execution
    - No runtime errors

Observation

- Apex class encapsulated business logic successfully.
- Trigger delegated logic to the Apex class.
- Developer Console executed Apex code without errors.
- Debug logs confirmed correct execution flow.

Result: Advanced Apex logic was implemented using an Apex class and trigger, and the code was tested and debugged successfully using Salesforce Developer Console.

Conclusion: Separating business logic into Apex classes improves maintainability and reusability. Salesforce Developer Console is an essential tool for testing and debugging Apex code in real-time.

PRACTICAL – 10

Title: Build a Custom Lightning App using Lightning App Builder and customize the application interface.

Aim: To create and customize Lightning pages using Salesforce Lightning App Builder by designing a custom Home Page, Record Page, and App Page for the Sales application.

Software / Tools Required
• Web Browser (Google Chrome recommended)
• Internet Connection
• Salesforce Developer Edition Account

Related Trailhead Learning (For Concept Understanding Only)
Trailhead Module: Lightning App Builder
Trailhead Link:
https://trailhead.salesforce.com/content/learn/modules/lightning_app_builder
Note to Students:
Trailhead modules are provided only for conceptual learning. All execution, screenshots, and evaluation must be performed in Salesforce Developer Edition.

Theory
Lightning App Builder is a declarative customization tool in Salesforce that allows administrators and developers to create custom user interfaces using a drag-and-drop approach. It helps design Lightning pages without writing code.
Salesforce Lightning App Builder allows creation of three types of pages:
1. Home Page
Displays user-specific information such as tasks, events, and performance metrics.
2. Record Page
Defines how records of a specific object appear in Lightning Experience.
3. App Page
Displays information related to a specific Salesforce application using multiple components.
Using Lightning App Builder improves usability, provides quick access to important data, and enhances user productivity.

Practical Exercise

Part 1: Create a Custom Home Page
Task 1: Create a Home Page
Steps

1. Login to Salesforce Developer Edition.
2. Click the **Gear Icon** and select **Setup**.
3. In the **Quick Find** box, type **Lightning App Builder**.
4. Click **Lightning App Builder**.
5. Click **New**.
6. Select **Home Page** and click **Next**.

Enter the following details:

Field	Value
Label	Sales Home
API Name	Sales_Home

7. Select **Standard Home Page Template**.
8. Click **Done**.

Task 2: Add Components to the Home Page
Drag the following components from the **Components Pane**.

Component	Suggested Region
Performance	Left Region
Assistant	Right Region
Today's Tasks	Right Region
Key Deals	Bottom Region

After arranging the components:

Click **Save**.

Task 3: Activate the Home Page

1. Click **Activate**.
2. Select **App and Profile**.
3. Click **Assign to Apps and Profiles**.
4. Select **Sales App**.
5. Click **Next**.
6. Select **System Administrator Profile**.
7. Click **Next** and then **Save**.

Task 4: Verify the Home Page

1. Open **App Launcher**.
2. Select **Sales App**.
3. Navigate to the **Home Page**.

Verify the following components appear:
• Performance
• Assistant
• Today's Tasks
• Key Deals
✓ **Result:**
A custom Lightning Home Page was successfully created and assigned to the Sales App.

Part 2: Create a Custom Record Page
Objective
To create a customized Opportunity record page in Salesforce Lightning Experience.

Task 1: Create a Record Page
Steps

1. Go to **Setup**.
2. Search **Lightning App Builder** in Quick Find.
3. Click **Lightning App Builder**.
4. Click **New**.
5. Select **Record Page** and click **Next**.

Enter the following details:

Field	Value
Label	New Opportunity Page
Object	Opportunity

6. Select **Header and Right Sidebar Template**.
7. Click **Done**.

Task 2: Add Components

1. Drag **Highlights Panel** to the **Top Region**.
2. Drag **Tabs Component** to the **Main Region**.
3. Click **Save**.

Task 3: Configure Tabs
Select the **Tabs Component** and ensure the following tabs exist:

Tab	Component
Activity	Activities
Details	Record Detail
Related	Related Lists

Click **Save**.
Then click **Activate**.

Task 4: Verify the Record Page

1. Open the **Sales App**.
2. Navigate to **Opportunities**.
3. Open any **Opportunity Record**.

Verify the following components:
• Highlights Panel
• Activity Tab
• Record Detail Section
• Related Lists

✓ **Result:**
A customized Opportunity Record Page was successfully created and activated.

Part 3: Build an App Lightning Page
Objective
• Add components to a Lightning App Page
• Configure properties of a Lightning page
• Add global actions to the page
• Add the page to Lightning Experience navigation

Task 1: Create an App Page
Steps

1. Go to **Setup**.
2. Search **Lightning App Builder**.
3. Click **New**.
4. Select **App Page** and click **Next**.

Enter the following details:

Field	Value
Label	Top Accounts and Opportunities

5. Select **Two Regions Template**.
6. Click **Done**.

Task 2: Add Components
List View Component
Drag **List View** to the first region.
Configure the properties:

Property	Value
Object	Account
Filter	All Accounts
Records to Display	5

Recent Items Component
Drag **Recent Items** to the second region.
Configure properties:

Property	Value
Custom Label	Recent Opportunities
Selected Object	Opportunity

Task 3: Add Page Actions

1. Select **Page** in the properties panel.
2. Click **Select Actions**.
3. Add the following actions:

• Log a Call
• New Account
• New Opportunity

4. Click **Save**.

Task 4: Activate the App Page

1. Click **Activation**.
2. Keep the default page name.
3. Change the icon to **Blue Lightning Bolt**.
4. Set **Tab Visibility: All Users**.

Add Page to Lightning Experience

1. Open **Lightning Experience Tab**.
2. Select **Sales App**.
3. Click **Add Page to App**.
4. Place the page in the **second position of the navigation bar**.

Click **Save**.

Task 5: Test the App Page

1. Open **App Launcher**.
2. Select **Sales App**.
3. Click **Top Accounts and Opportunities** in the navigation bar.

Verify:
• List View showing Account records
• Recent Opportunities section
• Global actions available
• Log a Call
• New Account
• New Opportunity

Observation
• Lightning App Builder allows customization of Salesforce user interfaces.
• Home Pages can display task and performance information.
• Record Pages help organize record information using components.
• App Pages combine multiple components to provide application-level insights.

Result: Custom Lightning pages including a Home Page, Record Page, and App Page were successfully created using Lightning App Builder.

Conclusion: Lightning App Builder enables administrators to customize Salesforce applications using a drag-and-drop interface. This improves usability, provides quick access to important data, and enhances overall productivity of Salesforce users.